Course 2 — Python Data Structures

CHAPTER 6

Strings in Python

What you will learn	String fundamentals & indexing • Slicing • Type conversion • Looping • Membership testing • Comparison • The String Library & its most useful built-in methods

SECTION 1 — THEORY

1. What is a String?

A **string** is one of Python's most fundamental built-in data types. At its core, a string is simply an **ordered, immutable sequence of characters**. Each character can be a letter, digit, punctuation mark, whitespace, or even a Unicode emoji — strings in Python 3 are fully Unicode-aware by default.

Think of a string like a row of labelled boxes in a warehouse: every box (character) has a fixed address (index), you can read any box instantly by its address, but you cannot change the contents of a box in place — you must create a brand-new row.

> ■ NOTE
> Immutability means once a string is created, its characters cannot be altered. Methods that appear to 'change' a string actually return a new string object.

1.1 String Literals

A **string literal** is the source-code representation of a string value. Python accepts four quoting styles:

Style	Syntax	Typical Use Case
Single quotes	`'hello'`	Short strings, avoids escaping double quotes
Double quotes	`"hello"`	Short strings, avoids escaping single quotes
Triple single	`'''multi-line'''`	Multi-line strings, docstrings
Triple double	`"""multi-line"""`	Multi-line strings, docstrings (PEP 257 standard)

1.2 String Concatenation

The **+** operator joins two strings end-to-end, producing a new string. This is called **concatenation**. You can also use the ***** operator to repeat a string a given number of times.

String Concatenation & Repetition

```
str1 = "Hello"
str2 = 'there'
bob  = str1 + str2
print(bob)          # Hellothere

# Repetition operator
line = '-' * 30
print(line)         # ------------------------------
```

■ CAUTION

Unlike numbers, '+' on strings does NOT add values — it glues them together. '3' + '4' gives '34', not 7. Always be mindful of your data types.

2. String Indexing

Because a string is a **sequence**, every character has a numeric position called an **index**. Python uses **zero-based indexing** — the first character is at index 0, the second at index 1, and so on.

Positive indices (forward) Negative indices (backward)

b	a	n	a	n	a
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

Python also supports **negative indexing**, which counts from the end of the string. Index **-1** refers to the last character, **-2** the second-to-last, and so forth. This is extremely handy when you need the tail of a string without knowing its exact length.

Positive & Negative Indexing

```
fruit = 'banana'

# Positive indexing
print(fruit[0])    # b
print(fruit[1])    # a

# Negative indexing
print(fruit[-1])   # a  (last character)
print(fruit[-6])   # b  (same as fruit[0])
```

■ **CAUTION**

Accessing an index beyond the string's length raises an IndexError. Always validate or use len() before accessing dynamic indices.

3. The len() Function

The built-in **len()** function returns the total number of characters in a string, including spaces, punctuation, and special characters. It is O(1) — regardless of string length, Python returns the result in constant time because it stores the size internally.

len() Function

```
s = 'Monty Python'
print(len(s))        # 12  (space counts!)

# Common pattern: access last character safely
last = s[len(s) - 1]
print(last)          # n

# Even cleaner with negative index
print(s[-1])         # n
```

4. Type Conversion with Strings

Python is a strongly typed language — you cannot mix types implicitly. You must explicitly convert between strings and numeric types using built-in constructors. This is a critical concept in data pipelines, where user input arrives as raw strings that need to be cast before computation.

Function	Direction	Example	Result
int()	String → Integer	`int('42')`	`42`
float()	String → Float	`float('3.14')`	`3.14`

str()	Number → String	`str(100)`	`'100'`
int(float())	Float String → Int	`int(float('3.7'))`	`3`

> ■ **CAUTION**
> int('3.14') raises a ValueError — you cannot convert a float-formatted string directly to int.
> First convert to float, then to int: int(float('3.14')).

5. Looping Through a String

Since strings are sequences, you can iterate through them character by character. Python provides two approaches: the **while loop** (index-based) and the **for loop** (iterator-based). The for loop is preferred in modern Python code because it is more readable, less error-prone (no off-by-one errors), and expressive.

for loop (Preferred)	while loop (Verbose)

`fruit = 'banana'`
`for letter in fruit:`
`    print(letter)`
	`index = 0`
	`while index < len(fruit):`
	`    letter = fruit[index]`
	`    print(letter)`
	`    index = index + 1`

■ *Cleaner • No index management • Pythonic*	■ *Manual index • Risk of infinite loop or off-by-one error*

6. String Slicing

Slicing is one of Python's most powerful string operations. It lets you extract a **substring** — a contiguous portion of a string — using the syntax **s[start : stop : step]**.

Syntax	Meaning
`s[start:stop]`	Characters from index start up to (but NOT including) stop
`s[:stop]`	From the beginning up to stop
`s[start:]`	From start to the end of the string
`s[:]`	A full copy of the string
`s[::step]`	Every step-th character (e.g., s[::2] = every other char)
`s[::-1]`	Reverse the entire string (very common Python idiom)

Slicing in Action — 'Monty Python'

```
s = 'Monty Python'

print(s[0:4])   # Mont      → indices 0,1,2,3 (4 excluded)
print(s[6:7])   # P         → just one character
print(s[:2])    # Mo        → from start up to index 2
print(s[8:])    # thon      → from index 8 to end
print(s[:])     # Monty Python  → full copy
print(s[::-1])  # nohtyP ytnoM  → reversed string
print(s[::2])   # MnyPto    → every other character
```

■ TIP
If stop exceeds the string length, Python does NOT raise an error — it simply stops at the last character. So s[0:9999] safely returns the full string.

7. The 'in' Operator — Membership Testing

The **in** keyword checks whether one string is a substring of another. It returns a Boolean (**True** or **False**). This is far more readable than manually scanning characters with a loop, and is used extensively in text parsing, search filtering, and data validation tasks in Data Science.

Membership Testing with 'in'

```
print('nan' in 'banana')      # True
print('xyz' in 'banana')      # False
print('Python' in 'Monty Python')  # True

# Practical: check for keyword in a log line
log = 'ERROR: connection timeout at 14:32'
if 'ERROR' in log:
    print('Alert! Error found in log.')
```

8. String Comparison

Python compares strings **lexicographically** — character by character using each character's Unicode code point (essentially its ASCII value for English text). All standard comparison operators (**==, !=, <, >, <=, >=**) work on strings.

Key rule: Uppercase letters have lower Unicode values than lowercase letters. For example, 'A' has code point 65 while 'a' has 97, so 'a' > 'A' evaluates to True. This means 'banana' < 'cherry' (because 'b' comes before 'c'), and 'Banana' < 'banana' (because 'B' has a lower code point than 'b').

Lexicographic String Comparison

```
word = 'cherry'

if word == 'banana':
    print('All right, bananas.')
elif word < 'banana':
    print(word, 'comes before banana alphabetically.')
elif word > 'banana':
    print(word, 'comes after banana alphabetically.')

# Output: cherry comes after banana alphabetically.

# Unicode code points
print(ord('a'))   # 97
print(ord('A'))   # 65 — uppercase is always smaller
```

■ TIP

For case-insensitive comparison, normalise both strings first: word.lower() == other.lower(). This is a very common real-world pattern.

9. The String Library — Built-in Methods

Every Python string object comes with a rich set of built-in methods. These are not standalone functions you import — they are **methods** attached to the string object itself and called using dot notation: **string.method(arguments)**. Crucially, because strings are **immutable**, none of these methods alter the original string. They always return a **new string**.

Method	Description	Example	Output
`str.upper()`	Converts all characters to uppercase.	`'hello'.upper()`	`'HELLO'`
`str.lower()`	Converts all characters to lowercase.	`'HELLO'.lower()`	`'hello'`
`str.capitalize()`	First char uppercase, rest lowercase.	`'hello world'.capitalize()`	`'Hello world'`
`str.strip()`	Removes leading & trailing whitespace.	`' hi '.strip()`	`'hi'`
`str.lstrip()`	Removes leading (left) whitespace.	`' hi '.lstrip()`	`'hi '`
`str.rstrip()`	Removes trailing (right) whitespace.	`' hi '.rstrip()`	`' hi'`
`str.replace(old,new)`	Replaces all occurrences of old with new.	`'banana'.replace('a','o')`	`'bonono'`
`str.find(sub)`	Returns the lowest index of sub; -1 if not found.	`'banana'.find('na')`	`2`

`str.startswith(s)`	Returns True if string starts with s.	`'Python'.startswith('Py')`	`True`
`str.endswith(s)`	Returns True if string ends with s.	`'test.py'.endswith('.py')`	`True`
`str.split(sep)`	Splits string on sep; returns a list.	`'a,b,c'.split(',')`	`['a','b','c']`
`str.join(iterable)`	Joins elements of iterable with string as separator.	`'-'.join(['a','b','c'])`	`'a-b-c'`
`str.center(w, char)`	Centers string within width w, padded with char.	`'hi'.center(10,'*')`	`'****hi****'`

SECTION 2 — PRACTICAL & CODING

The following notebook walkthrough covers all core string concepts with executable Python code. Each cell is explained in detail: its **purpose**, a **line-by-line breakdown**, the expected **output**, and the key **insight** to take away.

Cell [1] String Creation & Basic Properties

Purpose

Establish the fundamentals: how to define a string, confirm its type, and retrieve its length.

Cell [1]

```
# Cell 1 — String Creation & Basic Properties

s1 = 'Hello, Data Science!'
s2 = "Python is powerful"

print(type(s1))          # <class 'str'>
print(len(s1))           # 20
print(len(s2))           # 18
print(s1 + ' ' + s2)    # Concatenation
```

■ OUTPUT

```
<class 'str'>
20
18
Hello, Data Science! Python is powerful
```

Line-by-Line Breakdown

s1 = 'Hello, Data Science!' — Creates a string using single quotes and binds it to the variable s1. The comma, space, and exclamation mark are all characters counted in the length.

s2 = "Python is powerful" — Creates another string using double quotes. Both quote styles are equivalent; you choose based on whether the string itself contains quotes.

type(s1) — Confirms Python recognises the value as a str object. This is useful for debugging type-related bugs in data pipelines.

len(s1) — Counts every character including the comma and the space. 'Hello, Data Science!' has 20 characters.

s1 + ' ' + s2 — Concatenates three strings: the two variables and a literal space string ' ' between them. Without the middle ' ', the result would be 'Hello, Data Science!Python is powerful' — merged with no gap.

■ **Key Insight: len() counts characters, NOT words. Spaces are characters too. Always be explicit when adding separators during concatenation.**

Cell [2] Indexing — Positive & Negative

Purpose

Demonstrate how to access individual characters using both forward (positive) and backward (negative) indexing.

Cell [2]

```
# Cell 2 — String Indexing

fruit = 'banana'

# Positive indexing
print(fruit[0])   # First character
print(fruit[1])   # Second character
print(fruit[5])   # Last character (index = len-1)

# Negative indexing
print(fruit[-1])  # Last character
print(fruit[-2])  # Second from last
print(fruit[-6])  # Same as fruit[0]
```

■ OUTPUT

```
b
a
a
a
n
b
```

Line-by-Line Breakdown

fruit[0] — Zero-based indexing means the first character is always at index 0. For 'banana', index 0 is 'b'.

fruit[5] — The string 'banana' has 6 characters (len = 6), so the last valid positive index is 5. Accessing fruit[6] would raise an IndexError.

fruit[-1] — Negative index -1 always gives the last character regardless of string length. This is equivalent to fruit[len(fruit)-1] but far cleaner.

fruit[-6] — For a 6-character string, -6 wraps around to index 0 (the start). In general, negative_index + len(string) = equivalent_positive_index.

■ Key Insight: Negative indexing is one of Python's most elegant features. Use fruit[-1] instead of fruit[len(fruit)-1] — it's shorter, clearer, and works universally regardless of string length.

Cell [3] String Slicing

Purpose

Explore all major slicing patterns including start, stop, step, open-ended slices, and the reverse trick.

Cell [3]

```
# Cell 3 — String Slicing

s = 'Monty Python'

print(s[0:4])    # 'Mont'   — indices 0,1,2,3
print(s[6:7])    # 'P'      — single char via slice
print(s[:2])     # 'Mo'     — from start, stop at 2
print(s[8:])     # 'thon'   — from 8 to end
print(s[:])      # full copy of the string
print(s[::2])    # every other character
print(s[::-1])   # reversed string
print(s[6:100])  # beyond range — no error, stops at end
```

■ OUTPUT

```
Mont
P
Mo
thon
Monty Python
MnyPto
nohtyP ytnoM
Python
```

Line-by-Line Breakdown

s[0:4] — The slice runs from index 0 (inclusive) to index 4 (exclusive). Characters at positions 0, 1, 2, 3 are returned — that's 'M','o','n','t' → 'Mont'. The stop index is always excluded.

s[:2] — Omitting the start index defaults to 0. So this is identical to s[0:2].

s[8:] — Omitting the stop index means 'go to the end'. Python does not raise an error if the end is omitted.

s[::2] — The third parameter is the step. A step of 2 means 'take every second character'. Steps can be any integer — s[::3] gives every third character.

s[::-1] — A step of -1 means 'traverse backwards one character at a time'. With no start/stop, this reverses the entire string. This is the canonical Pythonic string-reversal idiom.

s[6:100] — The stop index exceeds the string length (12), but Python gracefully clamps it to the actual end rather than raising an error.

■ Key Insight: Slicing NEVER raises an IndexError even if indices are out of range — Python clamps them silently. But direct indexing (s[100]) WILL raise IndexError. s[::-1] is the fastest, most Pythonic way to reverse a string.

Cell [4] Looping Through Strings

Purpose

Compare the for loop and while loop approaches for iterating over a string, and demonstrate a practical character-counting use case.

Cell [4]

```
# Cell 4 — Looping Through Strings

fruit = 'banana'

# Approach 1: for loop (Pythonic & preferred)
print('--- for loop ---')
for letter in fruit:
    print(letter, end=' ')     # end=' ' avoids newline

# Approach 2: while loop (index-based)
print('\n--- while loop ---')
index = 0
while index < len(fruit):
    print(fruit[index], end=' ')
    index = index + 1

# Practical: Count vowels using a loop
vowels = 0
for char in 'Hello, Data Science!':
    if char in 'aeiouAEIOU':
        vowels += 1
print(f'\nVowel count: {vowels}')
```

OUTPUT

```
--- for loop ---
b a n a n a
--- while loop ---
b a n a n a
Vowel count: 8
```

Line-by-Line Breakdown

for letter in fruit: — Python's for-loop directly iterates over the string's characters one by one. The variable 'letter' takes on each character's value per iteration without needing an index at all.

print(letter, end=' ') — The end parameter overrides the default newline character, so all characters print on the same line separated by spaces.

while index < len(fruit): — The while loop checks the index is within bounds on every iteration. If you forget 'index = index + 1', you create an infinite loop — a common beginner bug.

if char in 'aeiouAEIOU': — Uses membership testing (the 'in' operator) inside the loop to check if each character is a vowel. Both upper and lowercase vowels are handled by the combined string.

vowels += 1 — Shorthand for vowels = vowels + 1. Each time we encounter a vowel, we increment our counter.

■ **Key Insight: The for loop is superior for string iteration in 99% of cases. Reserve the while loop for when you need explicit index manipulation (e.g., comparing adjacent characters or jumping by more than one position).**

Cell [5] Type Conversion

Purpose

Demonstrate safe type conversion between strings and numeric types, including a real-world user-input simulation.

Cell [5]

```
# Cell 5 — Type Conversion

# String to integer
age_str = '25'
age_int = int(age_str)
print(type(age_str), age_str)   # <class 'str'>  25
print(type(age_int), age_int)   # <class 'int'>  25
print(age_int + 5)              # 30  (numeric addition)

# String to float
price_str = '19.99'
price_flt = float(price_str)
print(price_flt * 2)            # 39.98

# Float string to int (two-step conversion)
val = int(float('3.99'))        # 3  (truncates, no rounding)
print(val)

# Number to string
score = 95
msg = 'Your score is: ' + str(score)
print(msg)
```

■ OUTPUT

```
<class 'str'>  25
<class 'int'>  25
30
39.98
3
Your score is: 95
```

Line-by-Line Breakdown

int(age_str) — Converts the string '25' to the integer 25. After this, arithmetic operations like age_int + 5 work as expected. Without conversion, '25' + 5 would raise a TypeError.

float(price_str) — Converts '19.99' to the float 19.99, enabling floating-point arithmetic. In data science, this is essential when loading CSV data where all values arrive as strings.

int(float('3.99')) — Converts a decimal string to a float first, then truncates (does NOT round) to integer. int('3.99') would fail with ValueError.

'Your score is: ' + str(score) — You cannot concatenate a string and an integer directly. str(score) converts 95 to '95', making concatenation valid.

■ Key Insight: In real data pipelines (pandas DataFrames, CSV readers, APIs), data almost always arrives as strings. Mastering type conversion is one of the most practically important Python skills for a Data Scientist.

Cell [6] String Methods — Built-in Library

Purpose

Demonstrate the most commonly used string methods with clear examples, showing that each method returns a new string without modifying the original.

Cell [6]

```
# Cell 6 — String Methods

text = '  Hello, World!  '

# Case methods
print(text.upper())         # '  HELLO, WORLD!  '
print(text.lower())         # '  hello, world!  '
print(text.capitalize())    # '  hello, world!  ' → only 1st char

# Whitespace removal
print(text.strip())         # 'Hello, World!'
print(text.lstrip())        # 'Hello, World!  '
print(text.rstrip())        # '  Hello, World!'

# Search & Replace
print(text.find('World'))   # 9 (index of 'W' after strip)
print(text.replace('World', 'Python'))  # replaces all occurrences

# Split and Join
csv_row = 'Alice,30,Engineer'
parts = csv_row.split(',')  # ['Alice', '30', 'Engineer']
print(parts)
rejoined = ' | '.join(parts)
print(rejoined)             # Alice | 30 | Engineer

# Original string is unchanged (immutability!)
print(text)                 # '  Hello, World!  '
```

■ OUTPUT

```
   HELLO, WORLD!
   hello, world!
   hello, world!
 Hello, World!
 Hello, World!
   Hello, World!
 9
   Hello, Python!
 ['Alice', '30', 'Engineer']
 Alice | 30 | Engineer
   Hello, World!
```

Line-by-Line Breakdown

text.strip() — Removes all leading and trailing whitespace. This is arguably the most-used string method in data cleaning — raw data often has accidental spaces around values.

text.find('World') — Returns the starting index of the first occurrence of 'World' within text. Returns -1 if the substring is not found. Note: the leading spaces shift the index.

text.replace('World', 'Python') — Replaces ALL occurrences (not just the first) by default. To limit replacements, pass a third argument: text.replace('l', 'L', 1) replaces only the first 'l'.

csv_row.split(',') — Splits the string at every comma, returning a list. This is foundational for parsing CSV (Comma-Separated Values) data — a format ubiquitous in Data Science.

' | '.join(parts) — The join method is the inverse of split. It concatenates all elements in a list, using the string it's called on as the separator. This is much more efficient than looping and concatenating.

print(text) — The final print proves that the original text variable is completely unchanged. Every method returned a new string — the original was never mutated.

■ **Key Insight: strip() + split() + lower() are the holy trinity of data cleaning for text columns in pandas. Nearly every real-world text preprocessing pipeline uses these three methods extensively.**

Cell [7] Mini Project — Log Line Parser

Purpose

Bring together all concepts — indexing, slicing, in operator, comparison, and string methods — in a realistic mini-project: parsing a server log line to extract structured information. This mirrors real Data Engineering tasks.

Cell [7] — Mini Project

```
# Cell 7 — Mini Project: Log Line Parser

log = '2024-01-15 14:32:07 ERROR  Connection timeout for user=admin'

# 1. Extract date and time using slicing
date = log[:10]
time = log[11:19]
print(f'Date: {date}')     # 2024-01-15
print(f'Time: {time}')     # 14:32:07

# 2. Detect log level using 'in'
level = 'UNKNOWN'
for lvl in ['ERROR', 'WARNING', 'INFO', 'DEBUG']:
    if lvl in log:
        level = lvl
        break
print(f'Level: {level}')   # ERROR

# 3. Extract message part after the level
idx = log.find(level)
message = log[idx + len(level):].strip()
print(f'Message: {message}')

# 4. Check severity
if level == 'ERROR':
    print('ALERT: Critical issue detected!')
elif level < 'WARNING':   # lexicographic comparison
    print('Non-critical log entry.')
```

OUTPUT

```
Date: 2024-01-15
Time: 14:32:07
Level: ERROR
Message: Connection timeout for user=admin
ALERT: Critical issue detected!
```

Line-by-Line Breakdown

log[:10] — Extracts the date portion. Because log lines follow a fixed format (ISO 8601 date is always the first 10 characters), slicing is the perfect tool.

log[11:19] — Extracts the time portion. Index 10 is the space between date and time, so we start at 11. A time string HH:MM:SS is 8 characters, ending at 19.

for lvl in [...]: if lvl in log: — Iterates over possible log levels and uses the 'in' membership test to detect which level appears in the log line. The 'break' exits as soon as a match is found.

log.find(level) — Finds the starting index of the matched level string within the log line. This lets us dynamically locate where the message begins, regardless of the level's position.

log[idx + len(level):].strip() — Skips past the level string by advancing the index by its length, then strips any remaining whitespace to get the clean message. This combines slicing, len(), and strip() elegantly.

■ Key Insight: This mini-project shows how string operations compose naturally. Slicing extracts fixed-width fields; find() locates dynamic positions; in tests membership; strip() cleans up. Together, these are the foundation of log parsing, NLP preprocessing, and ETL pipelines in Data Engineering.

CHAPTER SUMMARY

Concept	Key Takeaway
What is a String	Immutable, ordered sequence of Unicode characters
Indexing	Zero-based; negative indices count from the end
len()	Returns character count including spaces; runs in O(1) time
Slicing [s:e:step]	Stop is exclusive; out-of-range gracefully clamps; s[::-1] reverses
Type Conversion	int(), float(), str() — critical for data pipelines
Looping	Prefer for loops; while loop only when index manipulation is needed
'in' Operator	Membership testing — fast, readable, returns True/False
Comparison	Lexicographic (Unicode order); use .lower() for case-insensitive compare
String Methods	Always return new strings; originals unchanged. strip+split+lower = data cleaning trio